

Consultas Avanzadas

Consulta	Descripción	Tiempo sin índice (s)	Tiempo con índice (s)	Mejora (%)	Tipo de operación optimizada	Observaciones
Consulta 1	JOIN entre usuario y credencial filtrando por ultimo_cambio en los últimos 30 días.	0.32	0.08	75%	JOIN + ORDER BY	El índice compuesto (ultimo_cambio, id_usuario) permitió pasar de ALL a range, reduciendo la lectura completa de la tabla.
Consulta 2	Conteo de usuarios agrupados por estado con condición HAVING.	1.95	1.24	36.4%	GROUP BY + HAVING	El índice en id_usuario mejoró la agrupación
Consulta 3	Subconsulta correlacionada que filtra usuarios con contraseñas viejas (>6 meses).	29.46	16.65	43.4%	Subconsulta + filtro temporal	El índice en fechaRegistro mejoró el acceso secuencial pero no optimizó totalmente la subconsulta.
Promedio general				51.6%		Las mejoras son notorias, especialmente en consultas con condiciones selectivas y JOIN directos.

Consulta 1 — JOIN + ORDER BY (75 % de mejora)

Esta consulta se benefició significativamente del índice compuesto (ultimo_cambio, id_usuario) porque MySQL pudo filtrar por rango de fechas y, al mismo tiempo, resolver el JOIN con la tabla usuario sin hacer un recorrido completo (ALL).

El tipo de acceso cambió a range, lo que indica que el motor utilizó el índice para ubicar rápidamente las filas correspondientes al rango temporal solicitado.

Causa de mejora: alta selectividad del filtro temporal (solo parte de los 300 000 registros cumplen la condición), lo que reduce drásticamente la cantidad de lecturas.

Consulta 2 — GROUP BY + HAVING (36,4 % de mejora)

La mejora fue más moderada. Aunque el índice en id_usuario permitió ordenar y agrupar más eficientemente, la consulta realiza un GROUP BY sobre un conjunto de datos grande (todos los usuarios), por lo que MySQL sigue necesitando recorrer gran parte de la tabla para calcular los conteos.

Causa de mejora: el índice ayudó en la etapa de agrupación, pero la falta de un filtro

más restrictivo hizo que el índice no reduzca sustancialmente el volumen de datos procesados.

Consulta 3 — Subconsulta correlacionada (43,4 % de mejora)

Esta fue la consulta más exigente: con tiempos de 29 s sin índice y 16 s con índice. El índice en fechaRegistro mejoró el acceso secuencial a la tabla usuario, pero la subconsulta interna sigue siendo costosa porque se ejecuta múltiples veces (una por cada fila candidata del conjunto exterior).

Explain:

Consulta 1: JOIN con filtro de rango temporal

Antes del índice: el EXPLAIN mostraba type = ALL tanto para la tabla credencial como para usuario, indicando una búsqueda secuencial completa (MySQL recorría todas las filas para evaluar la condición ultimo_cambio > ...).

	id	select_type	table	type	possible_keys	key	key_len	ref
▶	1	SIMPLE	c	ALL	id_usuario,idx_credencial_id_usuario,idx_credencial_ultimoCambio_idUsuario	NULL	NULL	NULL
	1	SIMPLE	u	eq_ref	PRIMARY,idx_usuario_id	PRIMARY	8	usuariocredencial.c.id_usuario

Después del índice: cambió a type = range en credencial, evidenciando que el motor utilizó el índice compuesto (ultimo_cambio, id_usuario) para buscar solo los registros dentro del rango de fechas solicitado.

	id	select_type	table	type	possible_keys	key	key_len	ref
▶	1	SIMPLE	c	range	id_usuario,idx_credencial_id_usuario,idx_credencial_ultimoCambio_idUsuario	idx_credencial_ultimoCambio_idUsuario	5	NULL
	1	SIMPLE	u	eq_ref	PRIMARY,idx_usuario_id	PRIMARY	8	usuariocredencial.c.id_usuario

El índice redujo el volumen de lectura y permitió resolver más rápido el JOIN y el ORDER BY.

Consulta 2: Agrupación con GROUP BY y HAVING

Antes del índice: el EXPLAIN mostraba type = index, lo que indica que MySQL debía leer gran parte de la tabla para agrupar los datos.

	id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
▶	1	SIMPLE	u	index	PRIMARY,idx_usuario_id	idx_usuario_activo	1	NULL	297426	Using index; Using temporary; Using index
	1	SIMPLE	c	eq_ref	id_usuario,idx_credencial_id_usuario	id_usuario	8	usuariocredencial.u.id	1	Using index

Después del índice: se mantuvo type = index, el tipo de acceso no cambió significativamente.

	id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
▶	1	SIMPLE	u	index	PRIMARY,idx_usuario_id	idx_usuario_activo	1	NULL	297426	Using index; Using temporary; Using index
	1	SIMPLE	c	eq_ref	id_usuario,idx_credencial_id_usuario,idx_credencial_ultimoCambio_idUsuario	id_usuario	8	usuariocredencial.u.id	1	Using index

El índice fue útil para el ordenamiento, pero no modificó el plan de ejecución general.

Consulta 3: Subconsulta correlacionada

El EXPLAIN mantuvo los mismos tipos de acceso (range y eq_ref) y no mostró un cambio en las filas estimadas.

	id	select_type	table	type	possible_keys	key	key_len	ref
▶	1	PRIMARY	credencial	range	id_usuario,idx_credencial_id_usuario,idx_credencial_ultimoCambio_idUsuario	idx_credencial_ultimoCambio_idUsuario	5	NULL
	1	PRIMARY	u	eq_ref	PRIMARY,idx_usuario_id	PRIMARY	8	usuariocredencial.credencial.id_usuario

	id	select_type	table	type	possible_keys	key	key_len	ref
►	1	PRIMARY	credencial	range	id_usuario,idx_credencial_id_usuario,idx_crede...	idx_credencial_ultimoCambio_idUsuario	5	NULL
	1	PRIMARY	u	eq_ref	PRIMARY,idx_usuario_id	PRIMARY	8	usuariocredencial.credencial_id_usu

El índice no fue suficiente por la estructura de la consulta; la mejora observada en tiempo (43 %) se debe más a la reducción en lecturas de disco que a un cambio en el plan del optimizador.

Conclusión general:

Las mediciones confirman que la creación de índices adecuados reduce significativamente los tiempos de ejecución, especialmente cuando las condiciones son selectivas y las consultas involucran JOIN o filtros de rango.

Sin embargo, la optimización no depende solo de los índices: la estructura lógica de la consulta (por ejemplo, reemplazar subconsultas por JOIN) y la selectividad de los filtros son factores determinantes para alcanzar el máximo rendimiento.